

# AtCoder Library 移植模板

自包含可粘贴版·基于 [ac-library v1.6](#)

Build Time: September 2, 2026

## Contents

|          |                                          |          |
|----------|------------------------------------------|----------|
| <b>1</b> | <b>AtCoder Library (ACL) 移植</b>          | <b>2</b> |
| 1.1      | ACL <code>segtree</code> (单点修改)          | 3        |
| 1.1.1    | 模板本体                                     | 3        |
| 1.1.2    | 示例: 单点赋值 + 区间最大值                         | 5        |
| 1.1.3    | 示例: 单点修改 + 最大子段和                         | 5        |
| 1.1.4    | 示例: 第 $k$ 个 1 与左右边界                      | 5        |
| 1.1.5    | 示例: 含负数时找首个前缀和 $\geq x$                  | 6        |
| 1.2      | ACL <code>lazy_segtree</code> (区间修改)     | 7        |
| 1.2.1    | 模板本体                                     | 7        |
| 1.2.2    | 示例: 区间加 + 区间和                            | 10       |
| 1.2.3    | 示例: 仿射修改 + 一至三次幂和                        | 10       |
| 1.2.4    | 示例: 赋值/加 + 和、最值与位置                       | 10       |
| 1.2.5    | 示例: 最小代价位置的权值和                           | 11       |
| 1.2.6    | 示例: 权值线段树求最小众数                           | 11       |
| 1.3      | <code>z_algorithm</code> : Z 函数 (扩展 KMP) | 12       |

# 1 AtCoder Library (ACL) 移植

本文档收录 [AtCoder Library v1.6](#) 里值得直接搬进赛场的组件，逐个改成**自包含可粘贴版**：

- 去掉 `#ifndef` 头尾守卫与 `namespace atcoder`，去掉 `std::` 前缀（模板里默认 `using namespace std;`）。
- 原本 `#include "atcoder/internal_bit"` 提供的 `bit_ceil / countr_zero` 内联成结构体私有 `static`，不再依赖任何 ACL 头文件。
- 模板参数统一用**函数指针形式** (`S (*op)(S, S)`) 而非 C++17 的 `auto op`——写法与 C++14 兼容，且报错信息比 `auto + static_assert` 版本短得多。用 C++17 以上编译器时把 `S (*op)(S, S)`，`S (*e)()` 换成 `auto op`，`auto e` 也能编译，行为一致。

**为什么要移植：**ACL 只在 AtCoder 评测机上预装，ICPC / CCPC 赛场机没有 `atcoder/` 头文件目录，`#include <atcoder/segtree>` 直接 CE。本地开发（如 CMake 里 `include_directories(SYSTEM .../ac-library)`）可以照常用官方头，但打印稿上必须是能手抄进单文件的版本。

**全文共同约定：**区间一律  $[l, r]$  左闭右闭，合法位置  $0 \dots n$ （构造  $(n)$  时内部开  $n+1$  个叶子）。1-based 就只碰  $1 \dots n$ 、0-based 就只碰  $0 \dots n-1$ ，多出来的那格恒为 `e()` 不影响任何结果——两种习惯都不用换算，与主板子 `template-main` 各章一致。空区间写  $r = l - 1$ 。

**这是本移植版对官方 ACL 的语义改动：**官方 `atcoder/segtree` 用  $[l, r)$  左闭右开、合法位置  $0 \dots n - 1$ 。本文档在 `prod / apply / min_left` 入口处 `r++` 映射回内部的半开实现，`max_right` 取不到时返回  $l - 1$ （官方返回  $l$ ）。所以**照着官方文档抄调用**会差一位，以本文档的注释为准。

**正确性验证：**本文档代码块经 `template-check/` 下的 `acl_stress/run.sh` 与官方 ACL v1.6 及朴素暴力三方对拍（含 ASan / UBSan）通过。该脚本直接从本文档 `.tex` 抽 `minted` 块编译，改了代码不重跑就会暴露不一致。

## 1.1 ACL segtree (单点修改)

维护满足结合律的二元运算  $op$  与其单位元  $e$  (即么半群 monoid) 上的序列, 支持单点赋值、区间求积与线段树上二分, 单次操作均为  $O(\log n)$ 。

接口一览 (区间一律  $[l, r]$  左闭右闭;  $seg(n)$  的合法位置是  $0 \dots n$ , 共  $n + 1$  个叶子):

- $segtree<S, op, e> seg(n)$ : 全部初始化为  $e()$ 。
- $segtree<S, op, e> seg(v)$ : 由  $vector<S>$  在  $O(n)$  内建树。
- $set(p, x) / get(p)$ : 单点赋值 / 单点读取。
- $prod(l, r) / all\_prod()$ : 闭区间 / 全区间求积;  $r = l - 1$  表示空区间。
- $max\_right(l, f)$ : 返回最大的  $r$ , 使  $f(op(a_l, \dots, a_r))$  为真; 取不到时返回  $l - 1$ 。
- $min\_left(r, f)$ : 返回最小的  $l$ , 使  $f(op(a_l, \dots, a_r))$  为真; 取不到时返回  $r + 1$ 。

注意:  $op$  与  $e$  必须是自由函数或函数指针;  $e()$  必须是真单位元。  $max\_right/min\_left$  的谓词必须满足  $f(e()) == true$ , 且沿扩展方向只能从真变假一次。

### 1.1.1 模板本体

```
/* AtCoder Library v1.6 atcoder/segtree 自包含移植 + 闭区间
 * segtree(n) 合法位置 0..n; 区间 [l, r] 左闭右闭
 * 1-based 只碰 1..n; 0-based 只碰 0..n-1
 * 从 vector 建树时 v[i] 对应位置 i, 长度由 vector 决定
 * 空区间写 r == l - 1, prod 返回 e()
 */
template<class S, S (*op)(S, S), S (*e)()>
struct segtree {
public:
    segtree() : segtree(0) {}

    explicit segtree(int n)
        : segtree(vector<S>(n + 1, e())) {}

    explicit segtree(const vector<S> &v)
        : _n(int(v.size())) {
        size = (int) bit_ceil((unsigned) _n);
        log = countr_zero((unsigned) size);
        d = vector<S>(2 * size, e());
        for (int i = 0; i < _n; i++)
            d[size + i] = v[i];
        for (int i = size - 1; i >= 1; i--)
            update(i);
    }

    void set(int p, S x) {
        assert(0 <= p && p < _n);
        p += size;
        d[p] = x;
        for (int i = 1; i <= log; i++)
            update(p >> i);
    }

    S get(int p) const {
        assert(0 <= p && p < _n);
        return d[p + size];
    }

    S prod(int l, int r) const {
        r++;
        assert(0 <= l && l <= r && r <= _n);
        S sml = e(), smr = e();
```

```

    l += size, r += size;
    while (l < r) {
        if (l & 1) sml = op(sml, d[l++]);
        if (r & 1) smr = op(d[--r], smr);
        l >>= 1, r >>= 1;
    }
    return op(sml, smr);
}

```

```

S all_prod() const { return d[1]; }

```

```

template<class F>
int max_right(int l, F f) const {
    assert(0 <= l && l <= _n);
    assert(f(e()));
    if (l == _n) return _n - 1;
    l += size;
    S sm = e();
    do {
        while (l % 2 == 0) l >>= 1;
        if (!f(op(sm, d[l]))) {
            while (l < size) {
                l = 2 * l;
                if (f(op(sm, d[l]))) {
                    sm = op(sm, d[l]);
                    l++;
                }
            }
            return (l - size) - 1;
        }
        sm = op(sm, d[l]);
        l++;
    } while ((l & -l) != l);
    return _n - 1;
}

```

```

template<class F>
int min_left(int r, F f) const {
    r++;
    assert(0 <= r && r <= _n);
    assert(f(e()));
    if (r == 0) return 0;
    r += size;
    S sm = e();
    do {
        r--;
        while (r > 1 && (r % 2)) r >>= 1;
        if (!f(op(d[r], sm))) {
            while (r < size) {
                r = 2 * r + 1;
                if (f(op(d[r], sm))) {
                    sm = op(d[r], sm);
                    r--;
                }
            }
            return r + 1 - size;
        }
        sm = op(d[r], sm);
    } while ((r & -r) != r);
    return 0;
}

```

```

private:

```

```

int _n, size, log;
vector<S> d;

static unsigned bit_ceil(unsigned n) {
    unsigned x = 1;
    while (x < n) x *= 2;
    return x;
}

static int countr_zero(unsigned n) {
    return __builtin_ctz(n);
}

void update(int k) {
    d[k] = op(d[2 * k], d[2 * k + 1]);
}
};

```

### 1.1.2 示例：单点赋值 + 区间最大值

负数序列中，最大值的单位元必须是足够小的数，不能写成 0。

```

// ACL_EXAMPLE: seg_point_max
using S = ll;

S op_max(S a, S b) { return max(a, b); }
S e_max() { return -4e18; }

using SegMax = segtree<S, op_max, e_max>;

// vector<S> a(n + 1, e_max()); // 1-based
// SegMax seg(a);
// seg.set(p, x);
// ll ans = seg.prod(l, r);

```

### 1.1.3 示例：单点修改 + 最大子段和

节点保存总和、最大前缀、最大后缀和最大子段和。额外的 empty 让空区间成为真正单位元，也避免用哨兵参与加法。

```

// ACL_EXAMPLE: seg_max_subarray
struct S {
    ll sum, pre, suf, best;
    bool empty;
};

S op(S a, S b) {
    if (a.empty) return b;
    if (b.empty) return a;
    return {
        a.sum + b.sum,
        max(a.pre, a.sum + b.pre),
        max(b.suf, b.sum + a.suf),
        max({a.best, b.best, a.suf + b.pre}),
        false
    };
}

S e() { return {0, 0, 0, 0, true}; }
S make_leaf(ll x) { return {x, x, x, x, false}; }

```

```

using SegMaxSubarray = segtree<S, op, e>;

// vector<S> a(n + 1, e());
// FOR(i, 1, n) a[i] = make_leaf(x[i]);
// SegMaxSubarray seg(a);
// seg.set(p, make_leaf(x));
// ll ans = seg.prod(l, r).best;

```

### 1.1.4 示例：第 $k$ 个 1 与左右边界

当叶子只有 0/1 时，前缀计数具有单调性。max\_right 找到“计数仍小于  $k$ ”的最后位置，再加一就是第  $k$  个 1。

```

// ACL_EXAMPLE: seg_kth_boundary
using S = ll;
S op(S a, S b) { return a + b; }
S e() { return 0; }
using SegCount = segtree<S, op, e>;

int kth_one(const SegCount &seg, ll k) {
    if (k <= 0 || seg.all_prod() < k) return -1;
    return seg.max_right(1, [&](S cnt) {
        return cnt < k;
    }) + 1;
}

int farthest_right(const SegCount &seg, int l,
    ↪ ll x) {
    // 元素非负时，区间和 <= x 才沿右扩单调。
    return seg.max_right(l, [&](S sum) {
        return sum <= x;
    });
}

int farthest_left(const SegCount &seg, int r,
    ↪ ll x) {
    int l = seg.min_left(r, [&](S sum) {
        return sum <= x;
    });
    return max(1, l); // 1-based 时排除额外的 0
    ↪ 号单位元
}

```

### 1.1.5 示例：含负数时找首个前缀和 $\geq x$

有负数时不能拿区间和判定单调性；节点必须维护“这段能达到的最大前缀和” `pre`。若累计信息的 `pre < x`，说明还没遇到答案。

```
// ACL_EXAMPLE: seg_negative_prefix
struct S {
    ll sum, pre;
    bool empty;
};

S op(S a, S b) {
    if (a.empty) return b;
    if (b.empty) return a;
    return {a.sum + b.sum,
            max(a.pre, a.sum + b.pre), false};
}

S e() { return {0, 0, true}; }
S make_leaf(ll x) { return {x, x, false}; }
using SegPrefix = segtree<S, op, e>;

int first_prefix_at_least(
    const SegPrefix &seg, int l, int n, ll
    ↪ x) {
    if (seg.prod(l, n).pre < x) return -1;
    int last_bad = seg.max_right(l, [&](S cur) {
        return cur.empty || cur.pre < x;
    });
    return last_bad + 1;
}
```

## 1.2 ACL lazy\_segtree (区间修改)

在 `segtree` 的幺半群  $(S, op, e)$  上再定义作用幺半群  $(F, composition, id)$ , 即可支持区间修改、区间查询和树上二分, 单次操作均为  $O(\log n)$ 。

**五件套约束:** `op` 必须满足结合律; `e` 是其单位元; `mapping(f, x)` 表示把作用  $f$  施加到  $x$ ; `composition(f, g)` 表示“先  $g$  后  $f$ ”; `id` 是恒等作用。作用还必须满足对 `op` 的分配律。

**最大的坑:** `mapping` 拿不到区间边界。凡是更新公式需要长度, 都要把 `len` 存进  $S$ , 让它随 `op` 一起合并。

**接口:** 除 `segtree` 的 `set/get/prod/all_prod/max_right/min_left` 外, 新增 `apply(p, f)` 与闭区间 `apply(l, r, f)`。因为查询前可能需要下推标记, `get/prod` 与两个二分接口不再是 `const`。

### 1.2.1 模板本体

```
/* ACL atcoder/lazysegtree 自包含移植 + 闭区间
 * lazy_segtree(n) 合法位置 0..n; 区间 [l, r] 左闭右闭
 * 1-based 只碰 1..n; 0-based 只碰 0..n-1
 * 从 vector 建树时 v[i] 对应位置 i, 长度由 vector 决定
 * 空区间写 r == l - 1, prod 返回 e(), apply 直接返回
 */
template<class S, S (*op)(S, S), S (*e)(),
         class F, S (*mapping)(F, S),
         F (*composition)(F, F), F (*id)()>
struct lazy_segtree {
public:
    lazy_segtree() : lazy_segtree(0) {}

    explicit lazy_segtree(int n)
        : lazy_segtree(vector<S>(n + 1, e())) {}

    explicit lazy_segtree(const vector<S> &v)
        : _n(int(v.size())) {
        size = (int) bit_ceil((unsigned) _n);
        log = countr_zero((unsigned) size);
        d = vector<S>(2 * size, e());
        lz = vector<F>(size, id());
        for (int i = 0; i < _n; i++)
            d[size + i] = v[i];
        for (int i = size - 1; i >= 1; i--)
            update(i);
    }

    void set(int p, S x) {
        assert(0 <= p && p < _n);
        p += size;
        for (int i = log; i >= 1; i--) push(p >> i);
        d[p] = x;
        for (int i = 1; i <= log; i++) update(p >> i);
    }

    S get(int p) {
        assert(0 <= p && p < _n);
        p += size;
        for (int i = log; i >= 1; i--) push(p >> i);
        return d[p];
    }

    S prod(int l, int r) {
        r++;
        assert(0 <= l && l <= r && r <= _n);
        if (l == r) return e();
        l += size, r += size;
        for (int i = log; i >= 1; i--) {
            if ((l >> i) << i) != 1) push(l >> i);
            if ((r >> i) << i) != r)

```

```

        push((r - 1) >> i);
    }
    S sml = e(), smr = e();
    while (l < r) {
        if (l & 1) sml = op(sml, d[l++]);
        if (r & 1) smr = op(d[--r], smr);
        l >>= 1, r >>= 1;
    }
    return op(sml, smr);
}

S all_prod() { return d[1]; }

void apply(int p, F f) {
    assert(0 <= p && p < _n);
    p += size;
    for (int i = log; i >= 1; i--) push(p >> i);
    d[p] = mapping(f, d[p]);
    for (int i = 1; i <= log; i++) update(p >> i);
}

void apply(int l, int r, F f) {
    r++;
    assert(0 <= l && l <= r && r <= _n);
    if (l == r) return;
    l += size, r += size;
    for (int i = log; i >= 1; i--) {
        if ((l >> i) << i != l) push(l >> i);
        if ((r >> i) << i != r)
            push((r - 1) >> i);
    }
    {
        int l2 = l, r2 = r;
        while (l < r) {
            if (l & 1) all_apply(l++, f);
            if (r & 1) all_apply(--r, f);
            l >>= 1, r >>= 1;
        }
        l = l2, r = r2;
    }
    for (int i = 1; i <= log; i++) {
        if ((l >> i) << i != l) update(l >> i);
        if ((r >> i) << i != r)
            update((r - 1) >> i);
    }
}

template<class G>
int max_right(int l, G g) {
    assert(0 <= l && l <= _n);
    assert(g(e()));
    if (l == _n) return _n - 1;
    l += size;
    for (int i = log; i >= 1; i--) push(l >> i);
    S sm = e();
    do {
        while (l % 2 == 0) l >>= 1;
        if (!g(op(sm, d[l]))) {
            while (l < size) {
                push(l);
                l = 2 * l;
                if (g(op(sm, d[l]))) {
                    sm = op(sm, d[l]);

```

```

        l++;
    }
    }
    return (l - size) - 1;
}
sm = op(sm, d[l]);
l++;
} while ((l & -1) != 1);
return _n - 1;
}

template<class G>
int min_left(int r, G g) {
    r++;
    assert(0 <= r && r <= _n);
    assert(g(e()));
    if (r == 0) return 0;
    r += size;
    for (int i = log; i >= 1; i--)
        push((r - 1) >> i);
    S sm = e();
    do {
        r--;
        while (r > 1 && (r % 2)) r >>= 1;
        if (!g(op(d[r], sm))) {
            while (r < size) {
                push(r);
                r = 2 * r + 1;
                if (g(op(d[r], sm))) {
                    sm = op(d[r], sm);
                    r--;
                }
            }
            return r + 1 - size;
        }
        sm = op(d[r], sm);
    } while ((r & -r) != r);
    return 0;
}

private:
int _n, size, log;
vector<S> d;
vector<F> lz;

static unsigned bit_ceil(unsigned n) {
    unsigned x = 1;
    while (x < n) x *= 2;
    return x;
}

static int countr_zero(unsigned n) {
    return __builtin_ctz(n);
}

void update(int k) {
    d[k] = op(d[2 * k], d[2 * k + 1]);
}

void all_apply(int k, F f) {
    d[k] = mapping(f, d[k]);
    if (k < size)
        lz[k] = composition(f, lz[k]);
}

```

```

    }

    void push(int k) {
        all_apply(2 * k, lz[k]);
        all_apply(2 * k + 1, lz[k]);
        lz[k] = id();
    }
};

```

### 1.2.2 示例：区间加 + 区间和

区间长度必须存进  $S$ ；空出来的 0 号位置长度为 0，因此不会影响 1-based 查询。

```

// ACL_EXAMPLE: lazy_range_add_sum
struct S {
    ll sum;
    int len;
};
using F = ll;

S op(S a, S b) {
    return {a.sum + b.sum, a.len + b.len};
}
S e() { return {0, 0}; }
S mapping(F f, S x) {
    return {x.sum + f * x.len, x.len};
}
F composition(F f, F g) { return f + g; }
F id() { return 0; }

using SegAddSum = lazy_segtree<S, op, e, F,
    mapping, composition, id>;

// vector<S> a(n + 1, e());
// FOR(i, 1, n) a[i] = {x[i], 1};
// SegAddSum seg(a);
// seg.apply(l, r, add);
// ll ans = seg.prod(l, r).sum;

```

### 1.2.3 示例：仿射修改 + 一至三次幂和

用  $x \leftarrow ax + b$  统一表达乘法、加法与赋值：乘  $v$  是  $\{v, 0\}$ ，加  $v$  是  $\{1, v\}$ ，赋值  $v$  是  $\{0, v\}$ 。节点同时维护  $\sum x$ 、 $\sum x^2$ 、 $\sum x^3$ 。

```

// ACL_EXAMPLE: lazy_affine_moments
const ll MOD = 998244353;

ll mul_mod(ll a, ll b) {
    return (ll)((__int128)a * b % MOD);
}

struct S {
    ll s1, s2, s3;
    int len;
};
struct F {
    ll mul, add; // x <- mul * x + add
};

S op(S x, S y) {

```

```

    return {(x.s1 + y.s1) % MOD,
            (x.s2 + y.s2) % MOD,
            (x.s3 + y.s3) % MOD,
            x.len + y.len};
}
S e() { return {0, 0, 0, 0}; }

S mapping(F f, S x) {
    ll m = f.mul, a = f.add;
    ll m2 = mul_mod(m, m), m3 = mul_mod(m2, m);
    ll a2 = mul_mod(a, a), a3 = mul_mod(a2, a);
    ll old1 = x.s1, old2 = x.s2, old3 = x.s3;
    ll n = x.len % MOD;
    ll s1 = (mul_mod(m, old1) + mul_mod(a, n))
        ↪ % MOD;
    ll s2 = (mul_mod(m2, old2)
        + mul_mod(2 * mul_mod(m, a) % MOD,
        ↪ old1)
        + mul_mod(a2, n)) % MOD;
    ll s3 = (mul_mod(m3, old3)
        + mul_mod(3 * mul_mod(m2, a) % MOD,
        ↪ old2)
        + mul_mod(3 * mul_mod(m, a2) % MOD,
        ↪ old1)
        + mul_mod(a3, n)) % MOD;
    return {s1, s2, s3, x.len};
}

F composition(F f, F g) {
    // 先 g 后 f
    return {mul_mod(f.mul, g.mul),
            (mul_mod(f.mul, g.add) + f.add) %
            ↪ MOD};
}
F id() { return {1, 0}; }

using SegMoments = lazy_segtree<S, op, e, F,
    mapping, composition, id>;

// add v: seg.apply(l, r, {1, v});
// multiply: seg.apply(l, r, {v, 0});
// assign v: seg.apply(l, r, {0, v});

```

### 1.2.4 示例：赋值/加 + 和、最值与位置

这一份合并“扶苏的问题”和“最大值位置”板子。最大值并列时取最左位置；整段赋值后，最左最大值就是该段左端点。

```

// ACL_EXAMPLE: lazy_assign_add_stats
struct S {
    ll sum, mn, mx;
    int mx_pos, left, len;

```

```

};
struct F {
    bool has_assign;
    ll assign, add;
};

S op(S a, S b) {
    if (a.len == 0) return b;
    if (b.len == 0) return a;
    int pos = a.mx >= b.mx ? a.mx_pos :
        ↪ b.mx_pos;
    return {a.sum + b.sum, min(a.mn, b.mn),
            max(a.mx, b.mx), pos, a.left,
            a.len + b.len};
}

S e() { return {0, 0, 0, 0, 0, 0}; }

S mapping(F f, S x) {
    if (x.len == 0) return x;
    if (f.has_assign) {
        ll v = f.assign + f.add;
        return {v * x.len, v, v,
                x.left, x.left, x.len};
    }
    x.sum += f.add * x.len;
    x.mn += f.add;
    x.mx += f.add;
    return x;
}

F composition(F f, F g) {
    if (f.has_assign) return f;
    g.add += f.add;
    return g;
}

F id() { return {false, 0, 0}; }

using SegStats = lazy_segtree<S, op, e, F,
    mapping, composition, id>;

// leaf i: {x, x, x, i, i, 1}
// assign: seg.apply(l, r, {true, v, 0});
// add: seg.apply(l, r, {false, 0, v});

```

### 1.2.5 示例：最小代价位置的权值和

区间加只平移最小值，不改变哪些位置达到最小值，因此对应权值和保持不变。常用于只统计“当前代价最小”转移点的 DP。

```

// ACL_EXAMPLE: lazy_min_weight
const ll MOD = 998244353;

struct S {
    ll mn, weight;
    bool empty;
};

using F = ll;

S op(S a, S b) {
    if (a.empty) return b;
    if (b.empty) return a;
    if (a.mn < b.mn) return a;

```

```

    if (b.mn < a.mn) return b;
    return {a.mn, (a.weight + b.weight) % MOD,
        ↪ false};
}

S e() { return {0, 0, true}; }
S mapping(F f, S x) {
    if (!x.empty) x.mn += f;
    return x;
}

F composition(F f, F g) { return f + g; }
F id() { return 0; }

using SegMinWeight = lazy_segtree<S, op, e, F,
    mapping, composition, id>;

// 真实叶子必须显式 set 成 {cost, weight, false}
// seg.apply(l, r, delta);
// S best = seg.prod(l, r);

```

### 1.2.6 示例：权值线段树求最小众数

离散化后，每个叶子代表一个权值。区间加维护一段权值的出现次数；合并时先取最高频次，频次相同取较小权值。

```

// ACL_EXAMPLE: lazy_value_mode
struct S {
    ll cnt, value;
    bool empty;
};

using F = ll;

S op(S a, S b) {
    if (a.empty) return b;
    if (b.empty) return a;
    if (a.cnt != b.cnt) return a.cnt > b.cnt ?
        ↪ a : b;
    return a.value < b.value ? a : b;
}

S e() { return {0, 0, true}; }
S mapping(F f, S x) {
    if (!x.empty) x.cnt += f;
    return x;
}

F composition(F f, F g) { return f + g; }
F id() { return 0; }

using SegMode = lazy_segtree<S, op, e, F,
    mapping, composition, id>;

// leaf i: {0, compressed_values[i], false}
// seg.apply(left_value_id, right_value_id,
    ↪ delta);
// auto [frequency, value, unused] =
    ↪ seg.all_prod();

```

### 1.3 z\_algorithm: Z 函数 (扩展 KMP)

对长度  $n$  的串  $s$ , 返回数组  $z$ , 其中  $z[i]$  是  $s$  与其后缀  $s[i..n-1]$  的最长公共前缀长度, 约定  $z[0] = n$ 。总复杂度  $O(n)$ 。

**下标约定 (本节例外):** 字符串这节不套本文档开头的闭区间约定——它没有 `prod` 这类区间接口, 返回的是一个和输入等长的数组。下标一律 **0-based**,  $z$  的长度就是  $|s|$ , 与官方 `atcoder::z_algorithm` 完全一致, 照官方文档抄调用不会差位。

接口一览:

- `z_algorithm(const string &s)`: 字符串重载, 内部逐字符转成 `vector<int>` 再跑泛型版。
- `z_algorithm(const vector<T> &s)`: 泛型版,  $T$  只需支持 `==`。离散化后的 `vector<int>`、`vector<ll>`、`pair` 序列都能直接跑, 这是它比「只吃 `string`」的写法更值钱的地方。
- 空串返回空 `vector` (不会越界, 也不会返回 `{0}`)。

三个易错点:

1.  $z[0]$  是  $n$  而不是 0。统计「 $s$  在自身出现次数」之类时若把  $z[0]$  一并算入会多算一次; 反过来做 `exkmp` 时又必须靠它。
2. 拼接法求「 $a$  的每个后缀与  $b$  的 LCP」时, 分隔符必须是**两串都不会出现的字符** (小写串用 `'#'`, 读入含任意可见字符时改用 `'\1'` 这种控制符), 否则 LCP 会跨过分隔符, 答案偏大。
3. 泛型版比较的是  $T$  的 `==`。传 `vector<double>` 时浮点相等不可靠, 先离散化成整数再跑。

**示例一: 扩展 KMP 模板题** (洛谷 P5410, 求  $z(b)$  以及  $a$  的每个后缀与  $b$  的 LCP 数组  $p$ )。ACL 版没有单独的「两串」接口, 标准做法是拼 `b + '#' + a` 跑一遍:

```
/* 题意: 给 a, b, 输出
 *   z[i] = LCP(b, b[i..])      长度 |b|
 *   p[i] = LCP(b, a[i..])      长度 |a|
 * 按题目要求各输出一个带权异或和
 */
int main() {
    string a, b;
    cin >> a >> b;

    // 拼接法: 分隔符要保证两串里都不出现
    // (本题只有小写字母, '#' 安全)
    vector<int> zz = z_algorithm(b + '#' + a);

    int n = a.size(), m = b.size();

    // 前 m 位就是 b 自身的 z 数组
    vector<int> z(zz.begin(), zz.begin() + m);

    // 跳过 b 和分隔符, 后 n 位就是 p 数组
    // 因为分隔符挡着, 匹配长度天然 <= m, 不用
    // min 截断
    vector<int> p(zz.begin() + m + 1, zz.end());
}
```

```
ll ans1 = 0, ans2 = 0;
for (int i = 0; i < m; i++)
    ans1 ^= 1LL * (i + 1) * (z[i] + 1);
for (int i = 0; i < n; i++)
    ans2 ^= 1LL * (i + 1) * (p[i] + 1);
cout << ans1 << "\n" << ans2 << "\n";
}
```

**示例二: 找出模式串所有出现位置 + 最小整周期。**  
这两件事都是  $z$  的直接推论, 不用再写 KMP:

```
// 1) pat 在 txt 中的所有出现位置 (0-based 起点)
vector<int> find_all(const string &txt, const
    string &pat) {
    vector<int> z = z_algorithm(pat + '\1' +
    txt);
    int m = pat.size();
    vector<int> res;
    for (int i = 0; i + m <= (int) txt.size();
    i++)
        // 拼接串里 txt[i] 的下标是 m + 1 + i
        if (z[m + 1 + i] >= m) res.push_back(i);
    return res;
}

// 2) s 的最小整周期 (循环节, 整除 n)
//   z[i] + i == n 说明后缀 s[i..] 是 s 的
//   border,
//   对应周期 i; 取最小的这种 i 且 n % i == 0
int min_period(const string &s) {
    int n = s.size();
    vector<int> z = z_algorithm(s);
    for (int i = 1; i < n; i++)
        if (z[i] + i == n && n % i == 0) return
        i;
    return n; // 没有真周期, 整串自己就是周期
}
```

模板本体:

```
/* AtCoder Library v1.6 atcoder/string 的
 * z_algorithm 自包含移植
 * z[i] = s 与后缀 s[i..n-1] 的最长公共前缀长度,
 * 约定 z[0] = n
 * 下标 0-based, 返回长度 == 输入长度,
 * 与官方接口完全一致
 * 复杂度 O(n); 空串返回空 vector
 */
template<class T>
vector<int> z_algorithm(const vector<T> &s) {
    int n = (int)s.size();
    if (n == 0) return {};
    vector<int> z(n);

    // 先当 0 参与递推 (否则 j = 0 时 z[j]
    // 会自污染),
    // 循环结束再改回 n
    z[0] = 0;

    // j = 目前「匹配段右端最靠右」的那个起点,
    // 该段是 [j, j + z[j]]
    for (int i = 1, j = 0; i < n; i++) {

```

```

    int &k = z[i];

    // i 落在匹配段外 -> 从 0 暴力匹配;
    // 落在段内 -> 用对称位置 z[i-j] 当初值,
    // 但不能超出段的剩余长度 j + z[j] - i
    k = (j + z[j] <= i) ? 0 : min(j + z[j]
        ↪ - i, z[i - j]);

    // 暴力扩展 (右端不回退, 总次数线性)
    while (i + k < n && s[k] == s[i + k])
        ↪ k++;

    // 只有更靠右才换起点
    if (j + z[j] < i + z[i]) j = i;
}

z[0] = n;
return z;
}

// 字符串重载: 逐字符转 int 再跑泛型版
vector<int> z_algorithm(const string &s) {
    int n = int(s.size());
    vector<int> s2(n);
    for (int i = 0; i < n; i++) s2[i] = s[i];
    return z_algorithm(s2);
}

```

和主板子那份的关系: template-main 的字符串章里有一份 lambda 写法的 Z 函数, 用  $[l, r]$  维护最右匹配区间; 本节是 ACL 原版风格, 用起点  $j$  加段长  $z[j]$  表达同一个区间, 两者等价。ACL 版多的是泛型 `vector<T>` 重载——序列而非字符串的题 (如离散化后的数组求 LCP) 直接能用。